

开源软件基础

基于 z3 的约束求解

Zhilei Ren



OSCAR Team, School of Software, Dalian University of Technology

December 25, 2025



Outline

1 z3 简介

2 示例

z3 简介

概念

Z3 is a theorem prover from Microsoft Research. It is licensed under the MIT license^a.

^a<https://github.com/Z3Prover/z3>

约束求解的历史

- 1 SAT: The satisfiability problem
- 2 SMT: Satisfiability modulo theories

可满足问题

SAT

可满足性 (Satisfiability) 是用来解决给定的真值方程式, 是否存在一组变量赋值, 使问题为可满足。布尔可满足性问题 (Boolean satisfiability problem; SAT) 属于决定性问题, 也是第一个被证明属于 NP 完全的问题。此问题在计算机科学上许多领域的皆相当重要, 包括计算机科学基础理论、算法、人工智能、硬件设计等等。^a

^a<https://zh.wikipedia.org/zh-cn/>

Outline

1 z3 简介

2 示例

约束求解示例

今有雉兔同笼，上有三十五头，下有九十四足，问雉兔各几何？

约束求解示例

上置头，下置足，半其足，以头除足，以足除头，即得。

```
num_heads = 35
num_feet = 94
num_feet_half = num_feet // 2
num_rabbits = num_feet_half - num_heads
num_chicken = num_heads - num_rabbits
```

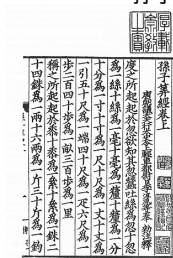
孙子算经

约束求解示例

上置头，下置足，半其足，以头除足，以足除头，即得。

```
num_heads = 35
num_feet = 94
num_feet_half = num_feet // 2
num_rabbits = num_feet_half - num_heads
num_chicken = num_heads - num_rabbits
```

孙子算经



约束求解示例

```
import z3
chicken, rabbits = z3.Ints('chicken rabbits')
z3.solve(chicken >= 1,      # number of chicken
         rabbits >= 1,      # number of rabbits
         chicken + rabbits == 35,
         chicken * 2 + rabbits * 4 == 94)
```

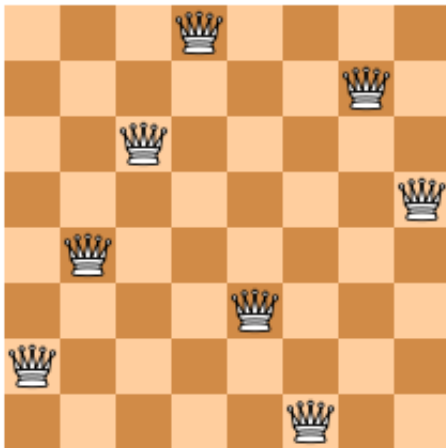
约束求解示例

$$\begin{aligned} \bigcirc + \bigcirc &= 10 \\ \bigcirc \times \square + \square &= 12 \\ \bigcirc \times \square - \triangle \times \bigcirc &= \bigcirc \\ \triangle &= ? \end{aligned}$$

约束求解示例

```
from z3 import *  
circle, square, triangle = Ints('circle square triangle')  
s = Solver()  
s.add(circle+circle==10)  
s.add(circle*square+square==12)  
s.add(circle*square-triangle*circle==circle)  
print(s.check())  
print(s.model())
```

八皇后问题



八皇后问题

```
Q = [ Int('Q_%i' % (i + 1)) for i in range(8) ]

# Each queen is in a column {1, ... 8 }
val_c = [ And(1 <= Q[i], Q[i] <= 8) for i in range(8) ]

# At most one queen per column
col_c = [ Distinct(Q) ]

# Diagonal constraint
diag_c = [ If(i == j,
             True,
             And(Q[i] - Q[j] != i - j, Q[i] - Q[j] != j - i))
          for i in range(8) for j in range(i) ]

solve(val_c + col_c + diag_c)
```

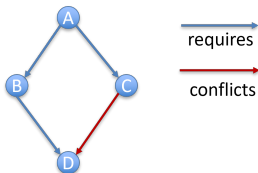
符号执行

```
def target_function(x, y):
    if x > 10:
        if y < 5:
            if x + y == 20:
                print("You found the secret!")
            else:
                print("Failed at math.")
        else:
            print("y is too large.")
    else:
        print("x is too small.")
from z3 import *
x = Int('x')
y = Int('y')
```

2. 创建一个求解器实例

```
solver = Solver()
solver.add(x > 10)           # 对应第一个 if
solver.add(y < 5)            # 对应第二个 if
solver.add(x + y == 20)      # 对应第三个 if
if solver.check() == sat:
    model = solver.model()
    print(f"推荐输入值: x = {model[x]}, y = {model[y]}")
```

Encoding Dependencies to Z3



```

import z3
A, B, C, D = z3.Bools("A B C D")
p1, p2, p3, p4, p5, p6 = z3.Bools("p1 p2 p3 p4 p5 p6")
solver = z3.Solver()
solver.assert_and_track(-1 * z3.If(A, 1, 0) + z3.If(B, 1, 0) >= 0, p1)
solver.assert_and_track(-1 * z3.If(A, 1, 0) + z3.If(C, 1, 0) >= 0, p2)
solver.assert_and_track(-1 * z3.If(A, 1, 0) + z3.If(D, 1, 0) >= 0, p3)
solver.assert_and_track(-1 * z3.If(B, 1, 0) + z3.If(D, 1, 0) >= 0, p4)
solver.assert_and_track(z3.If(C, 1, 0) + z3.If(D, 1, 0) <= 1, p5)
solver.assert_and_track(A == True, p6)

print(solver.check())
print(solver.unsat_core())
  
```